

1. Réflexions initiales technologiques

Pour ce projet j'ai choisi les technologies suivantes :

Symfony : J'ai choisi Symfony car c'est un framework PHP robuste et professionnel, très utilisé en entreprise de développement. Il propose une architecture MVC bien agencée, un système de sécurité complet et de nombreux bundles qui facilitent le développement.

Bootstrap 5 : J'ai choisi Bootstrap pour le front-end car il permet de créer rapidement des interfaces responsives sans écrire beaucoup de CSS. Il est aussi compatible avec tous les navigateurs et très bien documenté.

MySQL : J'ai choisi MySQL car c'est la base de données relationnelle la plus utilisée avec PHP. Elle est gratuite, performante et s'intègre bien avec Symfony via Doctrine ORM.

Doctrine ORM : Doctrine permet de manipuler la base de données via des objets PHP sans écrire de SQL manuellement. Il génère automatiquement les requêtes SQL à partir des entités.

VichUploaderBundle : Ce bundle Symfony facilite la gestion des uploads (téléchargement) de fichiers et images. Il gère automatiquement le stockage et la suppression des fichiers.

2. Configuration de l'environnement de travail

Outils installés :

- Windows 11
- VS Code avec extensions PHP et Twig
- XAMPP (serveur MySQL local)
- PHP 8.2
- Composer (gestionnaire de dépendances PHP)
- Symfony CLI (serveur de développement)
- HeidiSQL (gestion BDD MySQL)
- mailpit (pour gérer les emails en local)
- Git et GitHub (gestion des versions)

Installation du projet :

- Création du projet Symfony avec la commande : `symfony new traiteur --webapp`
 - Installation des bundles nécessaires via Composer
 - Configuration du fichier .env avec les paramètres de base de données
 - Création de la base de données avec Doctrine
-

3. Modèle Conceptuel de Données (MCD)

Les entités principales du projet sont :

- **Utilisateur** : id, email, password, nom, prenom, telephone, ville, pays, adresse_postale, role, actif
- **Menu** : id, titre, nombre_personne_minimum, prix_par_personne, regime, description, quantite_restante, theme_id
- **Plat** : id, titre_plat, note, photo
- **Commande** : id, numero_commande, date_commande, date_prestation, heure_livraison, prix_menu, prix_livraison, nombre_personne, statut, pret_materiel, retour_materiel, utilisateur_id, menu_id, adresse_prestation, ville_prestation, motif_annulation, mode_contact
- **CommandeStatut** : id, statut, date_changement, commande_id
- **Avis** : id, note, description, statut, utilisateur_id
- **Theme** : id, libelle
- **Regime** : id, libelle
- **Allergene** : id, libelle
- **MenuImage** : id, image_name, updated_at, menu_id

Relations :

- Un Menu a plusieurs Plats (ManyToMany)
- Un Menu a plusieurs Regimes (ManyToMany)
- Un Menu a un Theme (ManyToOne)
- Un Menu a plusieurs Images (OneToMany)
- Un Utilisateur a plusieurs Commandes (OneToMany)
- Une Commande a plusieurs CommandeStatuts (OneToMany)
- Un Plat a plusieurs Allergenes (ManyToMany)
- Un Utilisateur a plusieurs Avis (OneToMany)

4. Diagramme de cas d'utilisation

Acteurs :

- Visiteur (non connecté)
- Client (connecté)
- Employé
- Administrateur

Cas d'utilisation Visiteur :

- Consulter les menus
- Voir le détail d'un menu
- Consulter les avis
- Contacter le traiteur
- S'inscrire
- Se connecter

Cas d'utilisation Client :

- Passer une commande
- Modifier une commande

- Annuler une commande
- Suivre ses commandes
- Laisser un avis
- Modifier son profil

Cas d'utilisation Employé :

- Gérer les statuts des commandes
- Annuler une commande

Cas d'utilisation Administrateur :

- Gérer les menus
 - Gérer les plats
 - Gérer les utilisateurs
 - Gérer les avis
 - Gérer les allergènes
 - Consulter le chiffre d'affaires
 - Consulter les statistiques
-

5. Diagramme de séquence - Passer une commande

1. Le client se connecte à l'application
 2. Le client consulte la liste des menus
 3. Le client clique sur un menu pour voir le détail
 4. Le client clique sur "Commander ce menu"
 5. Le système affiche le formulaire de commande
 6. Le client remplit le formulaire (date, heure, nombre de personnes)
 7. Le client clique sur #Commander#
 8. Le système vérifie les données du formulaire
 9. Le système calcule le prix total et les frais de la livraison
 10. Le système enregistre la commande en base de données avec le statut #en_attente#
 11. Le système redirige le client vers son espace compte
 12. Le client voit sa commande dans la liste
-

6. Sécurité

Plusieurs mesures de sécurité ont été mises en place :

- **Authentification** : Symfony Security gère l'authentification avec hashage des mots de passe cryptés
- **Autorisation** : Les routes sont protégées avec des rôles (ROLE_USER, ROLE_EMPLOYE, ROLE_ADMIN)
- **Protection CSRF** : Tous les formulaires incluent un token CSRF pour éviter les attaques de faux formulaire
- **Validation des données** : Les formulaires sont validés côté serveur avant insertion en base de données

- **Variables d'environnement** : Les identifiants de connexion à la base de données sont disponibles dans le fichier README.md pour permettre aux correcteurs de tester l'application. En production, ces informations seraient stockées dans un fichier .env non versionné sur GitHub.
-

7. Documentation du déploiement

Déploiement en local :

1. Cloner le repository GitHub
2. Installer les dépendances avec `composer install`
3. Configurer le fichier .env avec les paramètres de base de données
4. Créer la base de données avec HeidiSQL
5. Importer le fichier `database/traiteur.sql`
6. Créer le dossier `public/uploads/menus`
7. Lancer le serveur avec `symfony server:start`
8. Accéder à l'application sur <http://localhost:8000>

Structure du projet :

- `src/Controller` : Controllers Symfony
 - `src/Entity` : Entités Doctrine
 - `src/Repository` : Repositories Doctrine
 - `src/Form` : Formulaires Symfony
 - `templates` : Vues Twig
 - `public/css` : Fichiers CSS
 - `public/js` : Fichiers JavaScript
 - `public/uploads` : Images Téléchargés
-